

C1 FlashKDA: 官方 kernel 为什么停在 SM80 MMA

B300 复现、瓶颈量化与 SM100 tcgen05 挑战

Assignment 02 · Team C1

2026

从“新指令应该更快”的直觉，走到“形状与依赖决定性能”的实证结论

核心问题

对 CHUNK=16、小矩阵密集、chunk 间带状态依赖的 FlashKDA，在 B300 上把 SM80 mma.sync 直接换成 SM100 tcgen05，是否更快？

直觉

- ▶ B300 支持更新的 tensor-core 指令
- ▶ tcgen05 有 TMEM 和更强峰值能力
- ▶ “新指令”似乎应该替代“旧指令”

实测答案

- ▶ 正确性：PASS
- ▶ 原版：1.755 ms
- ▶ tcgen05 改版：17.552 ms
- ▶ 改版约慢 10.001 倍

关键不是指令世代，而是算法粒度、物理 tile 与并行结构是否匹配。

KDA: 8192 个 token 为什么不能全部打散 ?

每个 head 的概念递推

$$S_t = d_t S_{t-1} + \Delta S_t, \quad o_t = q_t S_t$$

- ▶ 历史被压缩到 $D \times D$ 状态 S
- ▶ 同一 sequence、同一 head 的状态前后依赖
- ▶ FlashKDA 将序列分为长度 16 的 chunk

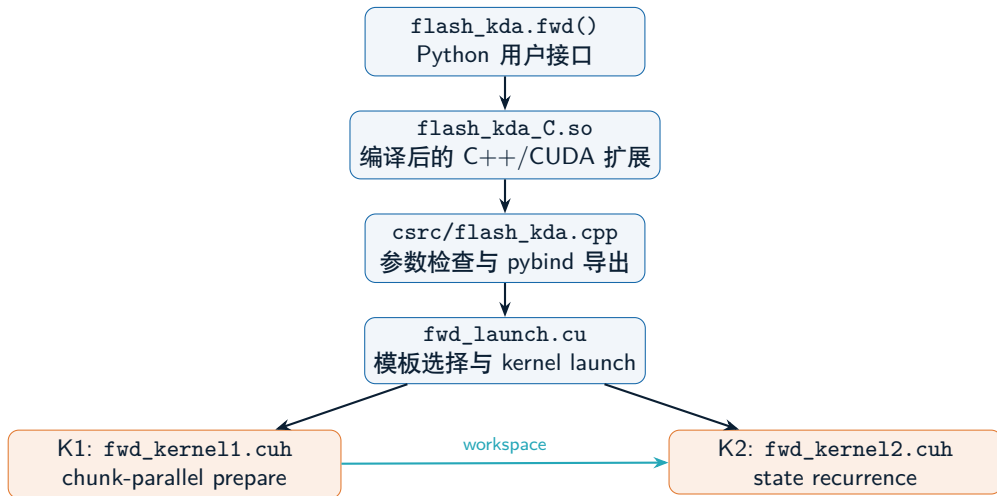


可并行: 不同 sequence、不同 head

不可直接并行: 同一 sequence/head

fixed $T = 8192, H = 96$: 每条链有 $8192/16 = 512$ 个 chunk, K2 只有约 $1 \times 96 = 96$ 条独立链。

从 Python 到 GPU: 项目结构与执行链



原版: `flash_kda` → `flash_kda_C`

改版: `flash_kda_tcgen05` →
`flash_kda_tcgen05_C`

C1 FlashKDA: 官方 kernel 为什么停在 C1420/MMA 处
两套命名允许在同一进程、同一输入、同一张卡上直接对拍